

HYBRIDRAG – HYBRID RETRIEVAL-AUGMENTED GENERATION FOR INTELLIGENT DOCUMENT SEARCH

2.AIM

To design and implement a **Hybrid Retrieval-Augmented Generation (Hybrid RAG) system** that combines **keyword-based retrieval using BM25** and **semantic retrieval using vector embeddings**, and uses **Reciprocal Rank Fusion (RRF)** to combine their results for accurate document question answering.

The system retrieves relevant information from PDF documents and uses a **locally hosted Qwen3 language model** to generate grounded answers based only on the retrieved document context.

3. OBJECTIVES

The major objectives of the HybridRAG system are:

1. To extract textual information from PDF documents.
2. To divide extracted text into smaller overlapping chunks.
3. To implement **BM25-based keyword retrieval** for exact and important terms.
4. To implement **semantic retrieval using sentence embeddings**.
5. To calculate similarity between user queries and document chunks using **cosine similarity**.
6. To combine BM25 and semantic retrieval using **Reciprocal Rank Fusion (RRF)**.
7. To provide the most relevant document chunks to a local language model.
8. To generate concise answers grounded in the uploaded document.
9. To display the source pages associated with generated answers.
10. To compare BM25, semantic and hybrid retrieval using **Precision, Recall and F1-score**.
11. To visualize the ranking contributions of BM25 and semantic retrieval in the hybrid ranking.

4. INTRODUCTION

Large language models are capable of generating fluent and informative responses, but they may not always have access to information contained in a user's private or newly uploaded documents. Retrieval-Augmented Generation addresses this limitation by retrieving relevant information from an external knowledge source before generating an answer.

Traditional retrieval methods and semantic retrieval methods have different strengths.

Keyword-based retrieval, such as BM25, performs well when the query contains important terms that appear directly in the document. However, it may fail when the query and document use different words to express the same meaning.

Semantic retrieval uses vector embeddings to identify content based on meaning rather than exact word matching. It can therefore retrieve semantically related information even when the wording differs.

HybridRAG combines both approaches. The system performs BM25 retrieval and semantic retrieval independently and then combines their rankings using **Reciprocal Rank Fusion**. The resulting hybrid ranking is passed to a local Qwen3 language model for answer generation.

The project is based on the concept of a **Hybrid RAG Search System**, where keyword and semantic retrieval are compared and combined, with BM25, embeddings, cosine similarity, hybrid retrieval, precision and recall forming the major technical concepts.

5. PROBLEM STATEMENT

A document search system based only on keyword matching may fail to retrieve relevant information when the query uses different terminology from the document.

Similarly, a system based only on semantic similarity may retrieve conceptually related passages while missing exact terms, names, technical expressions or important keywords.

Therefore, there is a need for a retrieval system that combines the advantages of both approaches.

The proposed **HybridRAG system** addresses this problem by combining:

- Keyword retrieval using **BM25**
- Semantic retrieval using **sentence embeddings**
- Ranking fusion using **Reciprocal Rank Fusion**
- Local answer generation using **Qwen3**

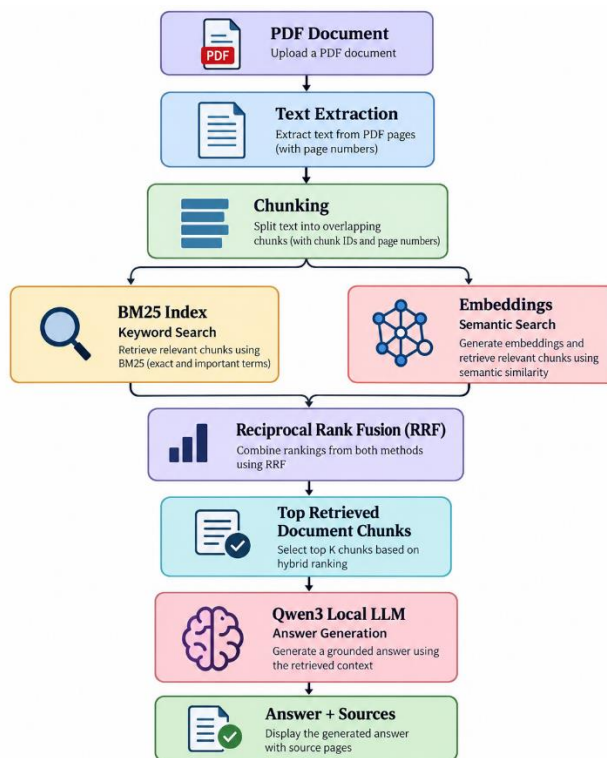
This enables the system to retrieve relevant document information using both lexical and semantic signals.

6. TECHNOLOGIES AND TOOLS USED

Technology / Tool	Purpose
Python	Core implementation
Streamlit	Web-based user interface
PyMuPDF	PDF text extraction
Sentence Transformers	Text embeddings
all-MiniLM-L6-v2	Embedding model
NumPy	Numerical computation
Rank-BM25	BM25 keyword retrieval
Ollama	Local LLM execution
Qwen3 1.7B	Answer generation
VS Code	Development environment
Scikit-learn / mathematical operations	Retrieval-related computation

7. SYSTEM ARCHITECTURE

The overall architecture of HybridRAG is:



8. MODULES OF THE SYSTEM

The HybridRAG system consists of the following major modules.

8.1 PDF Text Extraction

The uploaded PDF is processed using PyMuPDF. Text is extracted page by page and the corresponding page number is preserved.

Each extracted page is represented using:

Page Number

Page Text

This page information is later used to identify the source of retrieved information.

8.2 Document Chunking

Long document pages are divided into smaller chunks to make retrieval more effective.

The implemented chunking process:

1. Normalizes excessive whitespace.
2. Splits text into sentence-like units.
3. Combines sentences until the target chunk size is reached.
4. Maintains overlap between consecutive chunks.
5. Assigns a unique chunk ID.

6. Preserves the original page number.

The current system generated **14 chunks from the 6-page sample document** during testing.

9. BM25 KEYWORD RETRIEVAL

BM25 is a lexical information-retrieval algorithm used to rank documents based on the occurrence and importance of query terms.

In HybridRAG, every document chunk is tokenized and used to build a BM25 index.

When the user enters a query:

1. The query is converted into tokens.
2. BM25 calculates relevance scores for the document chunks.
3. Chunks are ranked according to their BM25 scores.
4. The top-ranked chunks are returned.

Advantages of BM25

- Effective for exact keyword matching.
 - Useful for technical terminology.
 - Does not require neural model inference.
 - Easy to interpret using retrieval scores.
-

10. SEMANTIC RETRIEVAL

Semantic retrieval attempts to identify document chunks based on their meaning.

HybridRAG uses the **all-MiniLM-L6-v2** sentence-transformer model to convert document chunks and user queries into numerical vectors.

The process is:

Document Chunk



Embedding Model



Vector Representation

The same process is applied to the user's question.

The similarity between the query vector and document vectors is calculated using **cosine similarity**.

Cosine Similarity

Cosine similarity measures the angle between two vectors.

The general formula is:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A higher similarity value indicates that the vectors are more closely aligned.

11. RECIPROCAL RANK FUSION

BM25 and semantic retrieval generate separate rankings.

Instead of directly comparing their raw scores, HybridRAG combines their **ranking positions** using Reciprocal Rank Fusion.

The RRF score is calculated as:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

where:

- d = document chunk
- $rank_r(d)$ = rank of the chunk in retrieval method r
- k = constant used to reduce the effect of high rankings

The implemented system uses:

$$k = 60$$

A chunk appearing in both retrieval lists receives contributions from both rankings.

Therefore, RRF allows BM25 and semantic retrieval to contribute complementary evidence to the final ranking.

12. HYBRID RETRIEVAL PROCESS

The hybrid retrieval process follows these steps:

1. Receive the user's question.
2. Perform semantic retrieval.
3. Perform BM25 keyword retrieval.
4. Obtain the top results from both methods.
5. Record the ranking position of each chunk.
6. Calculate the RRF score.
7. Combine chunks retrieved by either method.
8. Sort chunks according to their RRF score.
9. Select the top hybrid results.
10. Pass the selected context to Qwen3.

The system also displays:

- Hybrid Rank
- BM25 Rank
- Semantic Rank
- RRF Score

- Source Page
- Chunk ID

This allows the retrieval process to be inspected and analyzed.

13. GENERATION USING QWEN3

After retrieval, the most relevant chunks are provided to the locally running **Qwen3 1.7B** model.

The generation module instructs the model to:

- Use only the supplied document context.
- Avoid external information.
- Avoid inventing information.
- State when information is unavailable.
- Mention relevant source pages.
- Provide concise answers.

The use of a local model also allows the system to perform generation without depending on an external generative AI API.

14. WORKING PROCEDURE

The complete working procedure is:

Step 1 – Upload Document

The user uploads a PDF document through the Streamlit interface.

Step 2 – Extract Text

Text is extracted from each PDF page.

Step 3 – Create Chunks

The extracted text is divided into overlapping chunks.

Step 4 – Generate Embeddings

Each chunk is converted into a vector representation using the sentence-transformer model.

Step 5 – Build BM25 Index

A BM25 index is created from the document chunks.

Step 6 – Enter Query

The user enters a natural-language question.

Step 7 – Perform Retrieval

Two retrieval methods are executed:

- BM25 retrieval
- Semantic retrieval

Step 8 – Perform Hybrid Fusion

RRF combines the rankings generated by both retrieval methods.

Step 9 – Generate Answer

The top retrieved chunks are passed to Qwen3.

Step 10 – Display Result

The system displays:

- Generated answer
 - Source pages
 - BM25 results
 - Semantic results
 - Hybrid results
 - Ranking information
-

15. USER INTERFACE

The application is implemented using **Streamlit**.

The interface provides:

Document Settings

Allows the user to upload a PDF document.

Document Status

Displays:

- Number of pages
- Number of chunks
- Embedding size

Ask Your Document

Provides a text field for entering questions.

Answer

Displays the generated answer from Qwen3.

Sources

Displays the pages used to generate the answer.

Retrieval Details

Contains three tabs:

1. **BM25**
2. **Semantic**
3. **Hybrid**

The Hybrid tab additionally displays the BM25 rank, semantic rank and RRF score.

16. RETRIEVAL COMPARISON

The system compares the results of BM25 and semantic retrieval.

The comparison identifies:

Common Chunks

Chunks retrieved by both BM25 and semantic retrieval.

BM25-Only Chunks

Chunks retrieved by BM25 but not by semantic retrieval.

Semantic-Only Chunks

Chunks retrieved by semantic retrieval but not by BM25.

Hybrid Results

Chunks selected by the final RRF ranking.

This comparison demonstrates how the two retrieval methods can provide complementary results.

17. ALGORITHM

HybridRAG Algorithm

Algorithm: HybridRAG

Input:

PDF document D

User query Q

Output:

Grounded answer A

1. Extract text from D.
2. Divide extracted text into overlapping chunks.
3. Generate embeddings for all chunks.
4. Build BM25 index from chunks.
5. Receive query Q.
6. Perform semantic retrieval:
 $\text{SemanticResults} = \text{VectorSearch}(Q)$
7. Perform keyword retrieval:
 $\text{BM25Results} = \text{BM25Search}(Q)$

8. Apply Reciprocal Rank Fusion:

```
HybridResults = RRF(  
    SemanticResults,  
    BM25Results  
)
```

9. Select top K hybrid results.

10. Construct document context from retrieved chunks.

11. Pass context and query to Qwen3.

12. Generate grounded answer A.

13. Display:

A
Source pages
BM25 rankings
Semantic rankings
Hybrid rankings

14. End.

18. EVALUATION

The retrieval system was evaluated using:

- **Precision**
- **Recall**
- **F1-score**

A predefined evaluation dataset containing questions and manually identified relevant chunk IDs was used.

The evaluation compares:

1. BM25
2. Semantic retrieval
3. Hybrid retrieval

The implemented evaluation calculates the metrics for each query and then computes average values for each retrieval method.

19. RESULTS

The obtained retrieval evaluation results were:

Retrieval Method	Precision	Recall	F1-Score
------------------	-----------	--------	----------

BM25	0.150	0.688	0.244
------	-------	-------	-------

Semantic	0.175	0.750	0.280
----------	-------	-------	-------

Hybrid	0.225	1.000	0.363
--------	-------	-------	-------

Interpretation

The evaluation shows that the three retrieval approaches behave differently.

BM25 achieved a precision of **0.150** and recall of **0.688**.

Semantic retrieval achieved a precision of **0.175** and recall of **0.750**, showing that semantic similarity retrieved additional relevant content compared with BM25 for the predefined evaluation queries.

The **Hybrid retrieval** achieved a precision of **0.225**, recall of **1.000**, and F1-score of **0.363** for the predefined evaluation set.

The results demonstrate the usefulness of combining lexical and semantic retrieval signals through RRF.

Note: These values represent the predefined evaluation queries and manually identified relevant chunk IDs used in this project. They should not be interpreted as general performance measurements for all documents or queries.

20. SAMPLE OUTPUT

Example Query

Question:

What machine learning techniques are used?

System Output

The system retrieves relevant chunks using BM25 and semantic retrieval, combines their rankings using RRF, and passes the highest-ranked hybrid chunks to Qwen3.

The interface displays:

Answer

↓

Generated answer from Qwen3

Sources

↓

Relevant document pages

Retrieval Details

↓

BM25

Semantic

Hybrid

The Hybrid tab additionally displays:

Hybrid Rank

BM25 Rank

Semantic Rank

RRF Score

Page

Chunk

HybridRAG

localhost:8501

StopDeploy

Document Settings

Upload a PDF document

Upload

200MB per file • PDF

Retrieval Pipeline

Keyword Retrieval

BM25 finds exact and important terms.

Semantic Retrieval

Embeddings find semantically related content.

Hybrid Retrieval

RRF combines both retrieval methods.

Generation

HybridRAG

Hybrid Retrieval-Augmented Generation for intelligent document search

Document loaded: sample.pdf

Pages

6

Chunks

14

Embedding Size

384

Ask your document

Enter your question

What machine learning techniques are used?

Search & Generate Answer

Qwen is generating the answer...

Windows Taskbar

System Tray

HybridRAG

localhost:8501

Deploy

Document Settings

Upload a PDF document

Upload

200MB per file • PDF

Retrieval Pipeline

Keyword Retrieval

BM25 finds exact and important terms.

Semantic Retrieval

Embeddings find semantically related content.

Hybrid Retrieval

RRF combines both retrieval methods.

Generation

Answer

The machine learning techniques used are TF-IDF Vectorizer and a trained classifier (such as Logistic Regression / Random Forest).

Sources

Page 2, Page 4

Retrieval Details

BM25SemanticHybrid

Keyword-based retrieval using BM25

Rank 1

Page: 2

Chunk: 6

BM25 Score: 3.9806

rototypes and analytics applications

4. System Architecture The system combines two engines: A) Rule Engine B) Machine Learning Model (TF-IDF + Classifier)

Windows Taskbar

System Tray

Document Settings

Upload a PDF document

Upload

200MB per file • PDF

Retrieval Pipeline

Keyword Retrieval

BM25 finds exact and important terms.

Semantic Retrieval

Embeddings find semantically related content.

Hybrid Retrieval

RRF combines both retrieval methods.

Generation

Retrieval Comparison

Comparison of chunks retrieved by BM25 and Semantic Search.

Common Chunks

6, 9

Semantic Only

7, 8, 10

BM25 Only

4, 12, 13

Hybrid Results

6, 9, 10

Retrieval Evaluation

Evaluate BM25, Semantic, and Hybrid retrieval using Precision, Recall, and F1-score.

Run Retrieval Evaluation

Document Settings

Upload a PDF document

Upload

200MB per file • PDF

Retrieval Pipeline

Keyword Retrieval

BM25 finds exact and important terms.

Semantic Retrieval

Embeddings find semantically related content.

Hybrid Retrieval

RRF combines both retrieval methods.

Generation

Retrieval Evaluation

Evaluate BM25, Semantic, and Hybrid retrieval using Precision, Recall, and F1-score.

Run Retrieval Evaluation

Retrieval evaluation completed!

Retrieval Method	Precision	Recall	F1-Score
BM25	0.15	0.688	0.244
Semantic	0.175	0.75	0.28
Hybrid	0.225	1	0.363

Metrics are calculated from the predefined evaluation questions and their relevant chunk IDs.

21. ADVANTAGES

The major advantages of HybridRAG are:

1. Combines keyword and semantic retrieval.
2. Handles both exact terminology and semantic similarity.
3. Uses Reciprocal Rank Fusion to combine retrieval rankings.
4. Provides source-page information for retrieved content.

5. Uses a local language model for answer generation.
 6. Reduces dependence on external generative AI APIs.
 7. Provides retrieval transparency through ranking information.
 8. Supports evaluation using Precision, Recall and F1-score.
 9. Provides an interactive Streamlit interface.
 10. Can be extended to support multiple documents and persistent storage.
-

22. LIMITATIONS

The current implementation has the following limitations:

1. The current evaluation uses a predefined set of queries.
 2. Relevant chunk IDs are manually defined for evaluation.
 3. The current system primarily operates on the uploaded document.
 4. Query expansion is not currently implemented.
 5. The chunking strategy can be further improved to understand document sections.
 6. The local Qwen3 model has limited computational resources compared with larger language models.
 7. The current system does not use persistent vector storage.
 8. Production-level authentication and deployment configuration are not implemented.
 9. Retrieval quality may vary depending on document structure and chunk boundaries.
-

23. FUTURE ENHANCEMENTS

The following enhancements can be implemented in future versions:

1. Improved Query Processing

Query expansion, keyword normalization and query rewriting can be introduced to improve retrieval for short or ambiguous questions.

2. Advanced Section-Aware Chunking

The chunking mechanism can be extended to understand headings, sections, paragraphs and document structure.

3. Performance and Inference Optimization

Embedding generation, retrieval and local LLM inference can be optimized for faster response times.

4. Production-Ready Configuration

Improved configuration management, error handling, logging and security can be added.

5. Deployment Support

The application can be deployed as a cloud or enterprise document-question-answering service.

6. Multi-Document Retrieval

The system can be extended to retrieve information across multiple uploaded documents.

7. Persistent Vector Storage

A vector database can be integrated to persist embeddings and enable scalable retrieval without recomputing embeddings every time.

24. CONCLUSION

The **HybridRAG system** successfully demonstrates a retrieval-augmented generation pipeline that combines keyword-based and semantic retrieval.

The system extracts text from PDF documents, creates overlapping chunks, generates vector embeddings, builds a BM25 index, performs independent retrieval using both methods and combines their rankings using **Reciprocal Rank Fusion**.

The retrieved context is then provided to a locally hosted **Qwen3 1.7B** model to generate a grounded answer. The system also displays source pages and detailed retrieval information, making the retrieval process more transparent.

The retrieval evaluation using the predefined evaluation dataset produced an F1-score of **0.363 for Hybrid retrieval**, compared with **0.244 for BM25** and **0.280 for Semantic retrieval**. The hybrid method achieved a recall of **1.000** on this evaluation set.

Thus, the project demonstrates how combining lexical and semantic retrieval can form an effective foundation for document-based question answering.